

CS 189/289

Today's lecture outline

1. Finish CNNs
2. Other NN stuff

Assigned reading:

9.1.2 (no free lunch)

9.3.1 (Early stopping)

9.3.2 (Double descent)

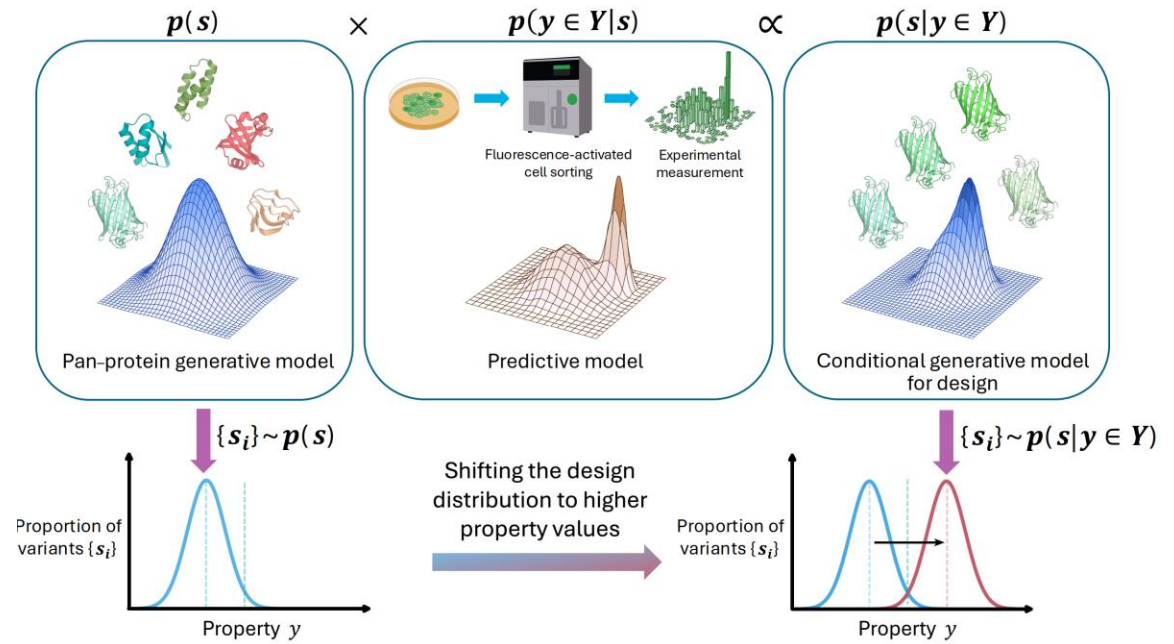
Hope to cover protein engineering in last lecture!

PROTEIN ENGINEERING

Science
AAAS

How artificial intelligence is reengineering protein engineering

Jennifer Listgarten^{1,2,3*} and Hanlun Jiang¹



coming out on Thursday

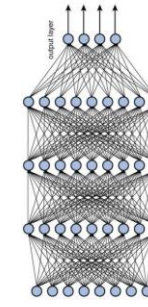
CS 189/289

Today's lecture outline

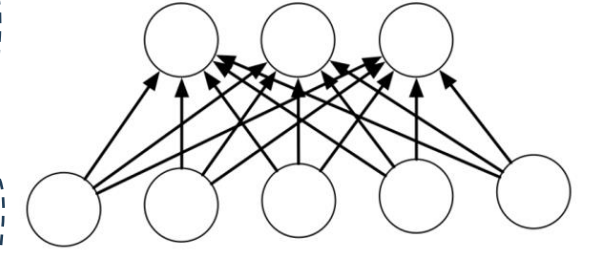
1. Finish CNNs
2. Other NN stuff

Recap: Convolutional NNs (CNNs)

Force the **weights to be shared**.

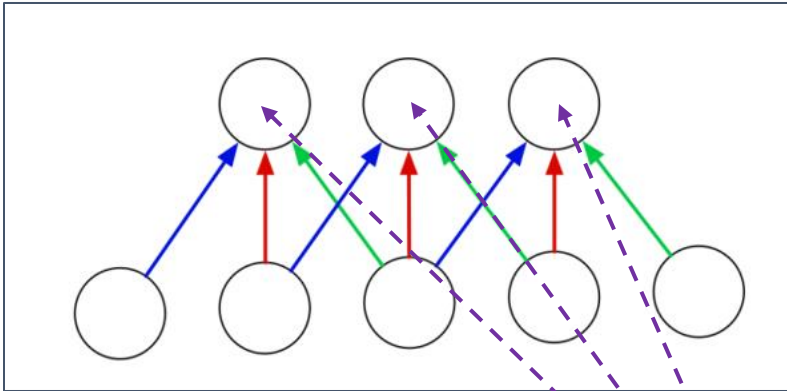


Fully-connected (FC) layer

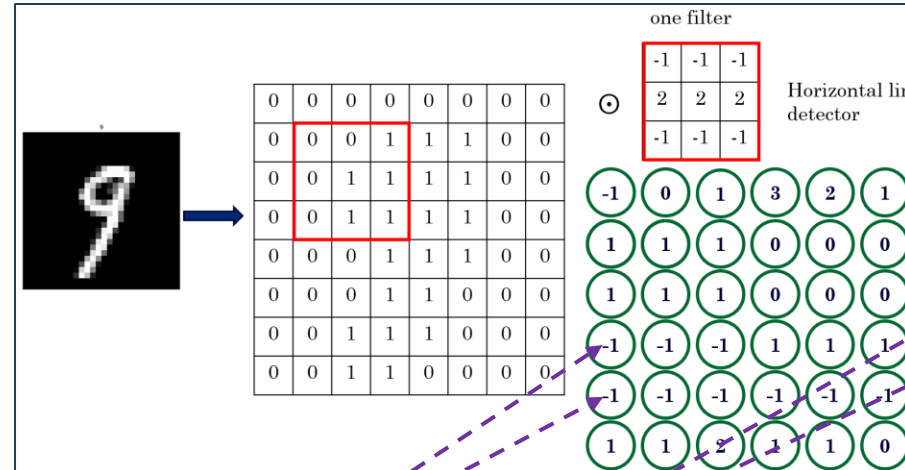


Three ways to depict one convolutional layer:

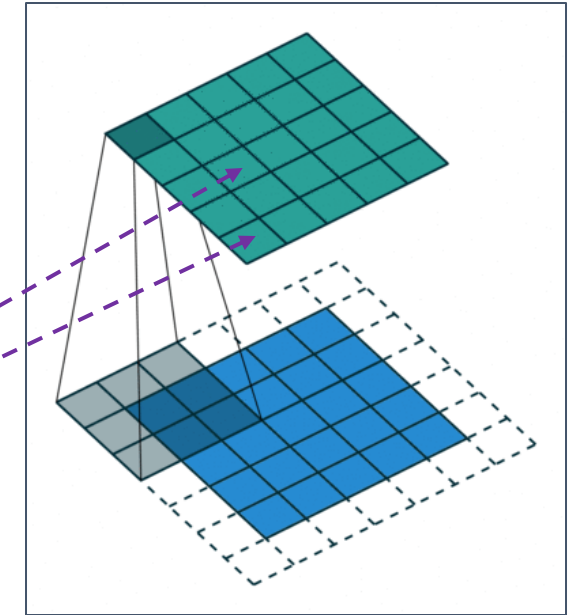
1



2



3



neurons ("hidden units") in a CNN layer

What If We Want to Keep the Same Image Dimensions?

- We can use padding.

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	1	1	0	0	0
0	0	0	1	1	1	1	0	0	0
0	0	0	1	1	1	1	0	0	0
0	0	0	0	1	1	1	0	0	0
0	0	0	0	1	1	0	0	0	0
0	0	0	1	1	1	0	0	0	0
0	0	0	1	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

Feature extractor

Or kernel

-1	-1	-1
2	2	2
-1	-1	-1

Horizontal line
detector

$\odot$

0

What If We Want to Keep the Same Image Dimensions?

- We can use padding.

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	1	1	0	0	0
0	0	0	1	1	1	1	0	0	0
0	0	0	1	1	1	1	0	0	0
0	0	0	0	1	1	1	0	0	0
0	0	0	0	1	1	0	0	0	0
0	0	0	1	1	1	0	0	0	0
0	0	0	1	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

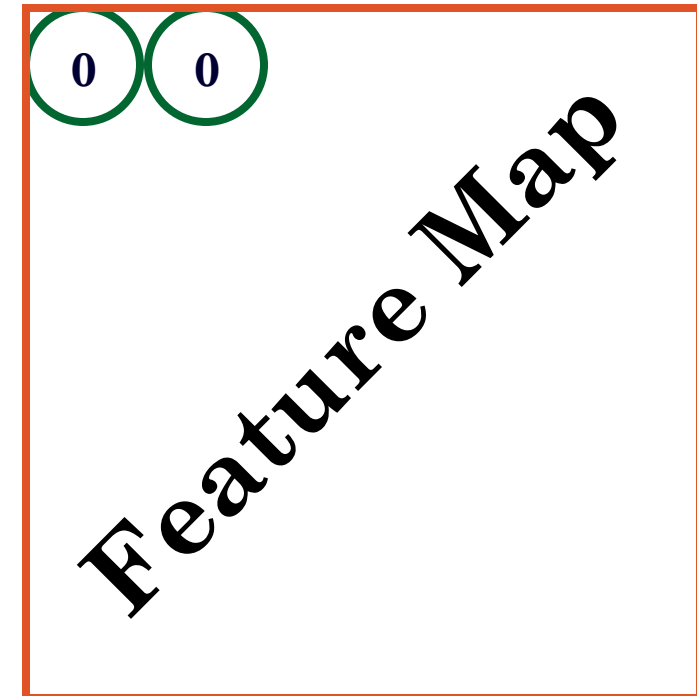
Feature extractor

Or kernel

-1	-1	-1
2	2	2
-1	-1	-1

$\odot$

Horizontal line detector



2D convolution

Can we get intuition about the numeric values of some kernels/filters?



*

1	0	-1
2	0	-2
1	0	-1

2D convolution

Can we get intuition about the numeric values of some kernels/filters?



*

1	0	-1
2	0	-2
1	0	-1



"vertical edges"

2D convolution

Can we get intuition about the numeric values of some kernels/filters?



*

0	-1	0
-1	4	-1
0	-1	0

2D convolution

Can we get intuition about the numeric values of some kernels/filters?



*

0	-1	0
-1	4	-1
0	-1	0



"sharpen"

2D convolution

But remember, in a neural network, we're going to learn these filters from the data, using gradient descent



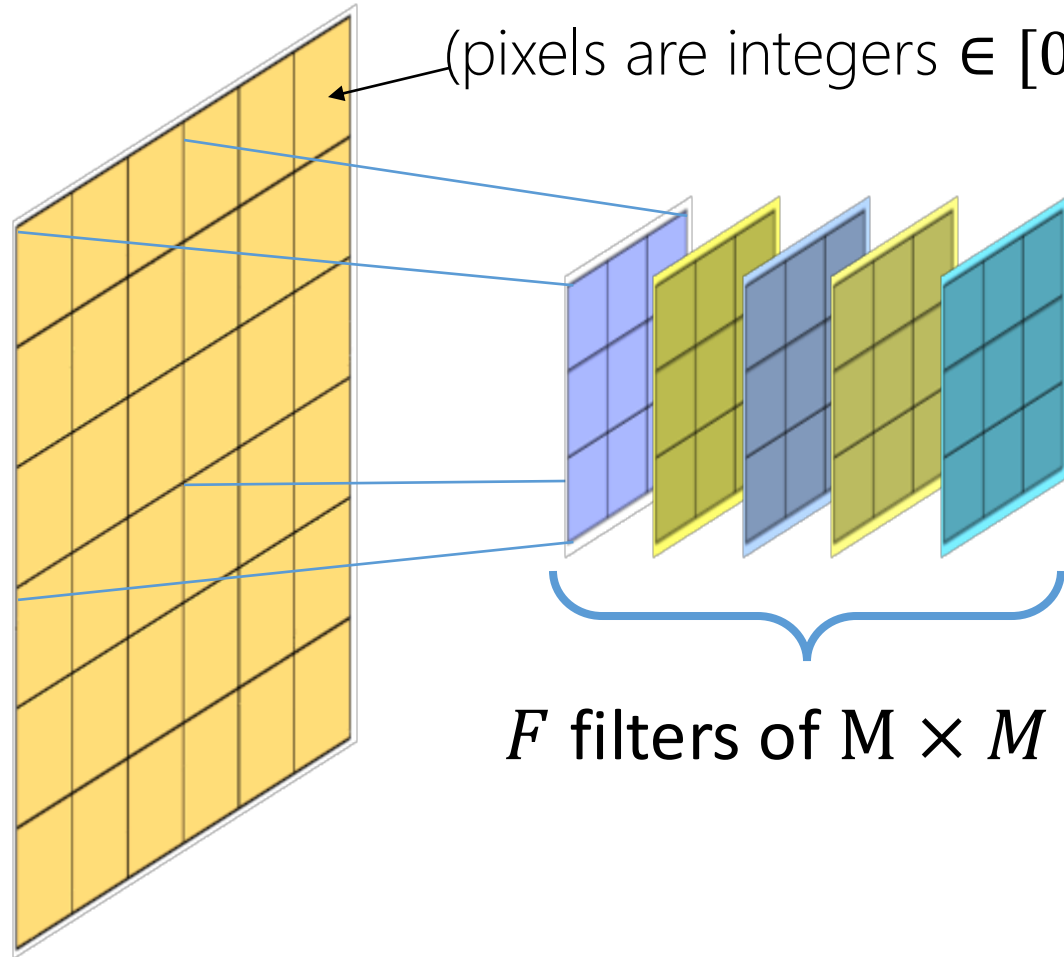
*

w_{11}	w_{12}	w_{13}
w_{21}	w_{22}	w_{23}
w_{31}	w_{32}	w_{33}

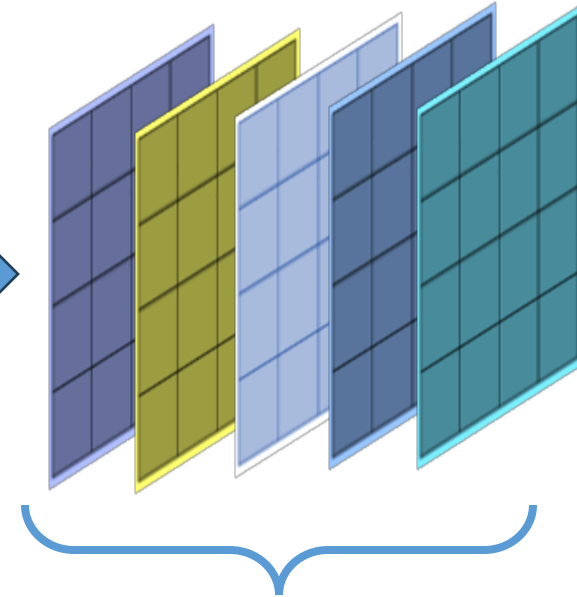


2D convolution of grayscale images

(pixels are integers $\in [0,255]$ for 8-bit representation)

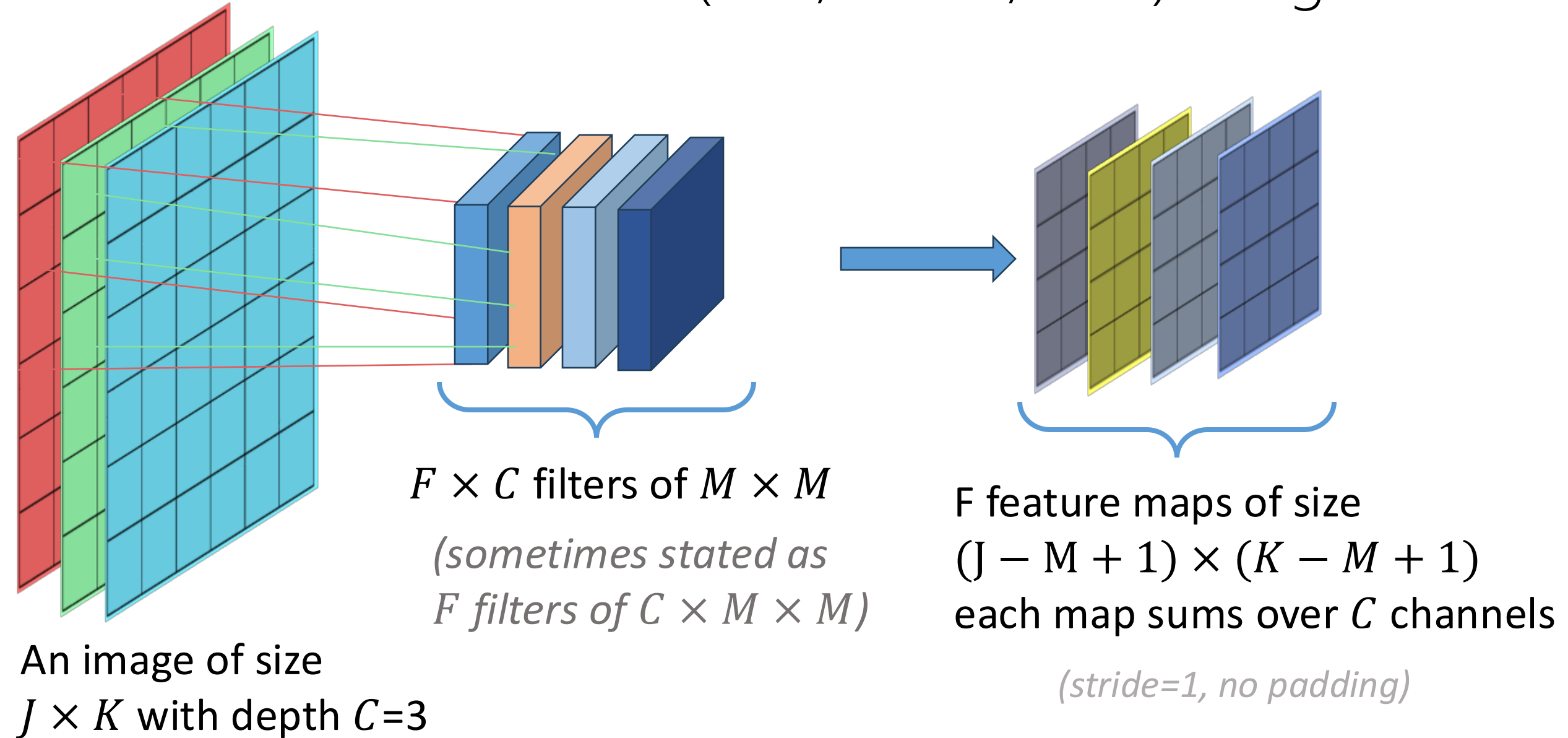


Grayscale image of
size $J \times K$

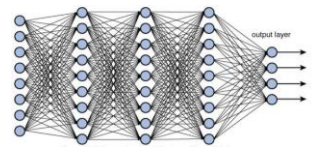


F feature maps of size
 $(J - M + 1) \times (K - M + 1)$
(*stride=1, no padding*)

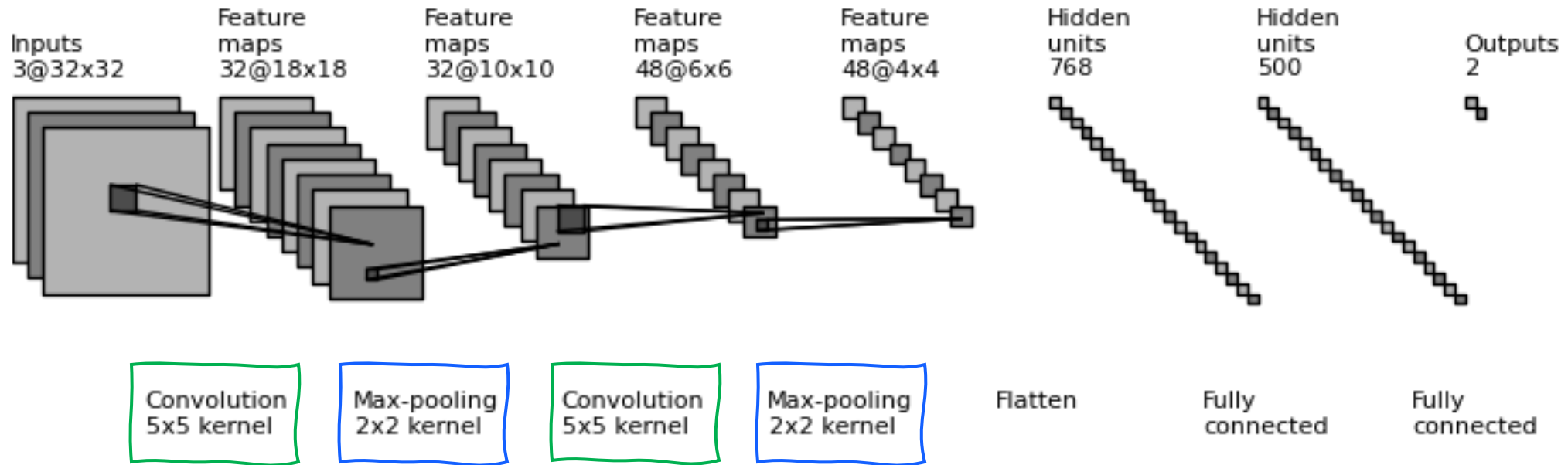
2D convolution of RGB (Red/Green/Blue) images



"Pooling layers": CNNs are more than just convolutional layers!

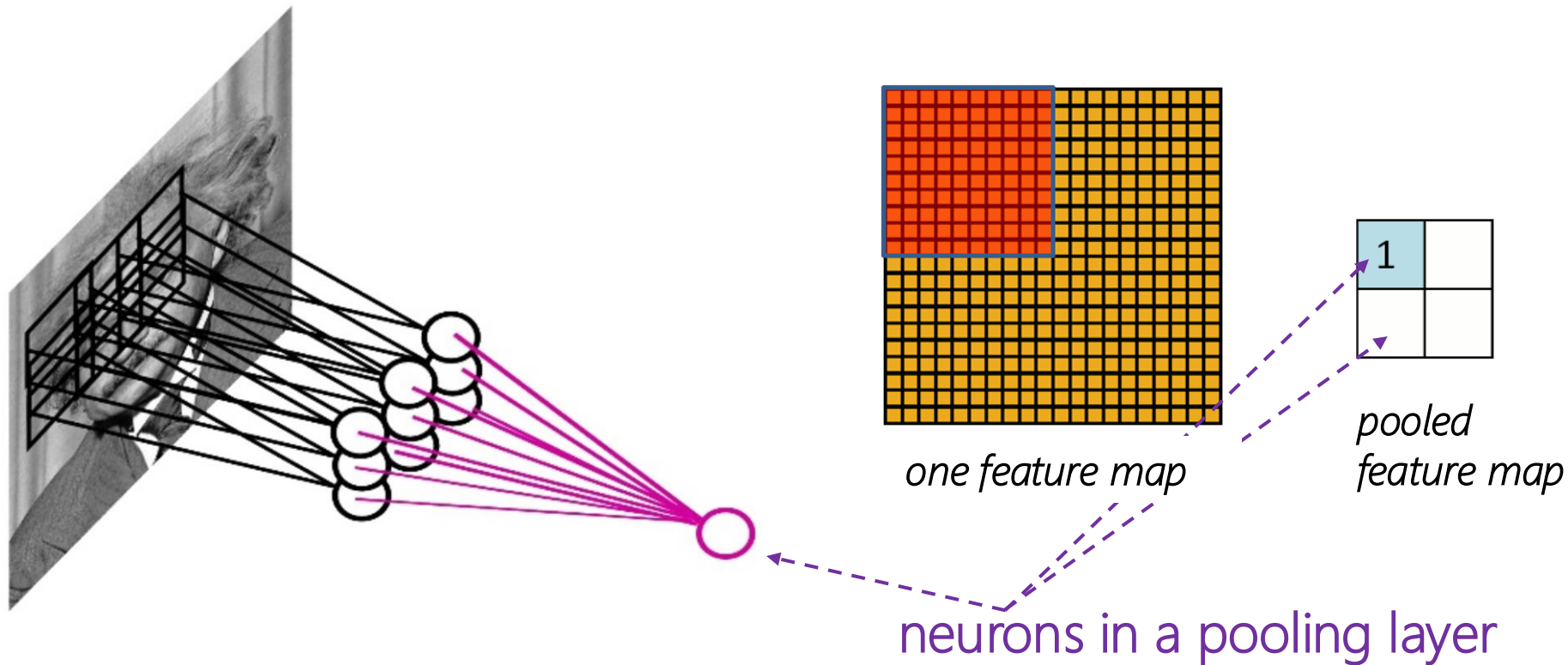


Fully-connected NN



Pooling layers (e.g., "max pooling")

- Reduces the size of the feature maps.
- Provides invariance to small transformations (e.g. if face is not perfectly centered when doing face detection).



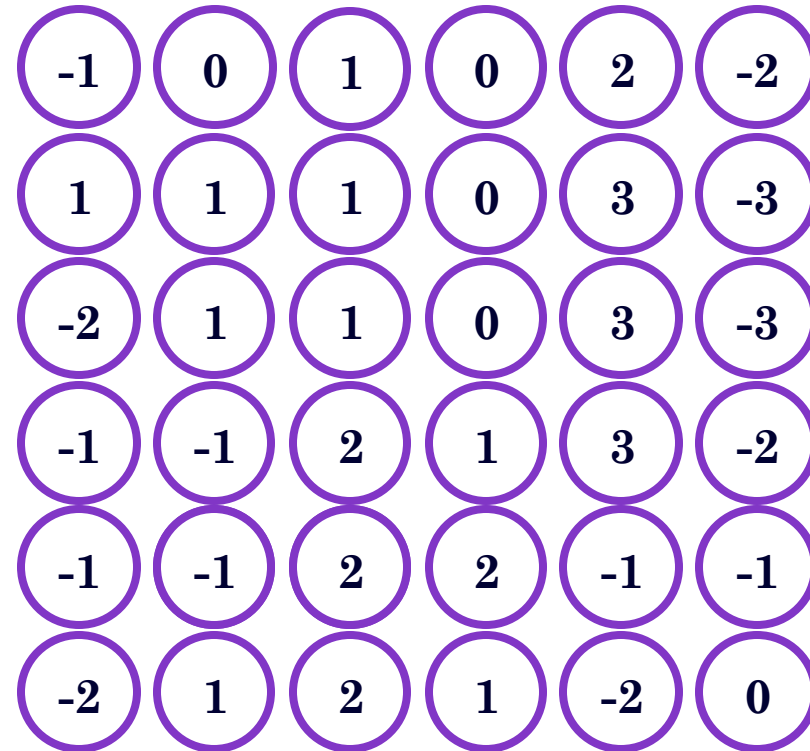
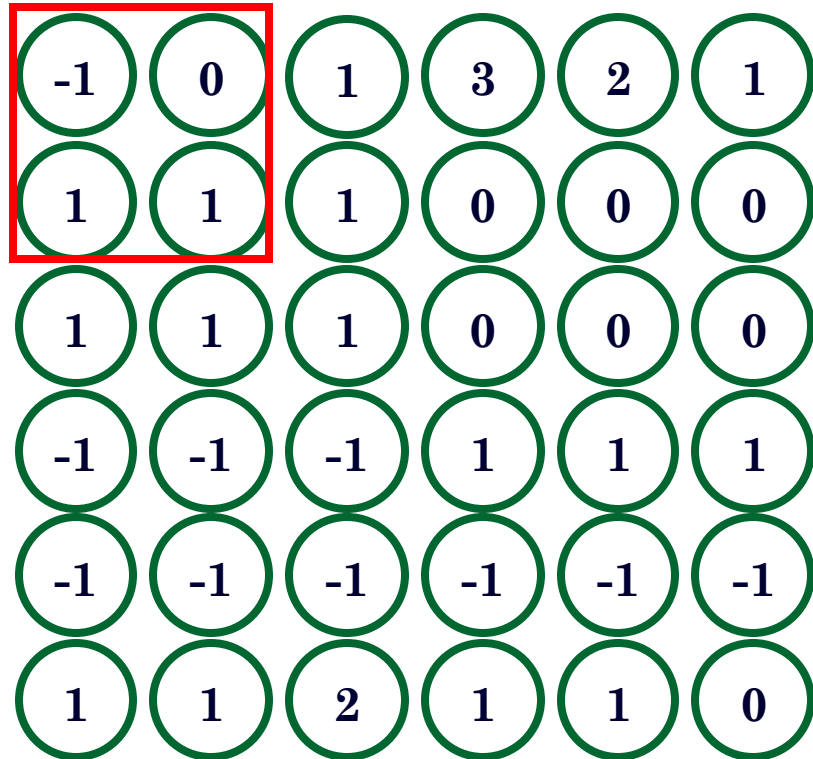
Pooling: downsampling feature maps

2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

“Max pooling”

-1	2	-1
-1	2	-1
-1	2	-1



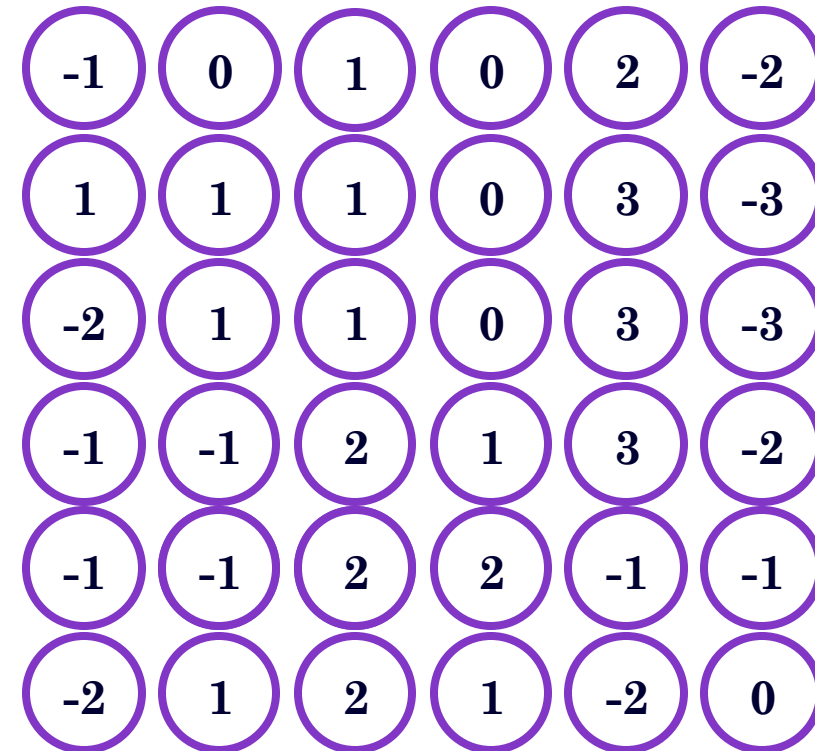
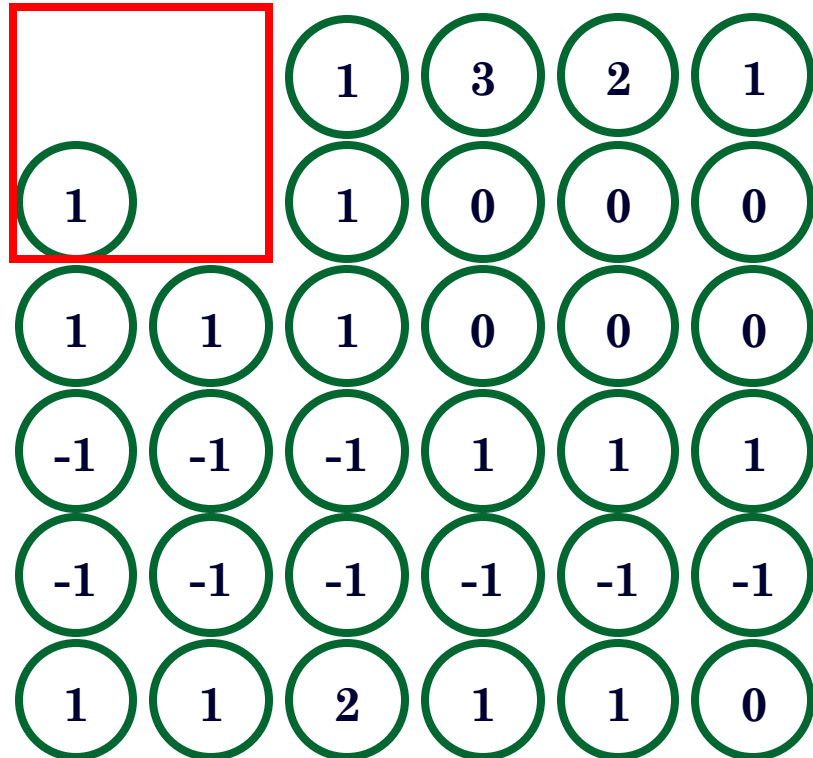
Pooling: downsampling feature maps

2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

“Max pooling”

-1	2	-1
-1	2	-1
-1	2	-1



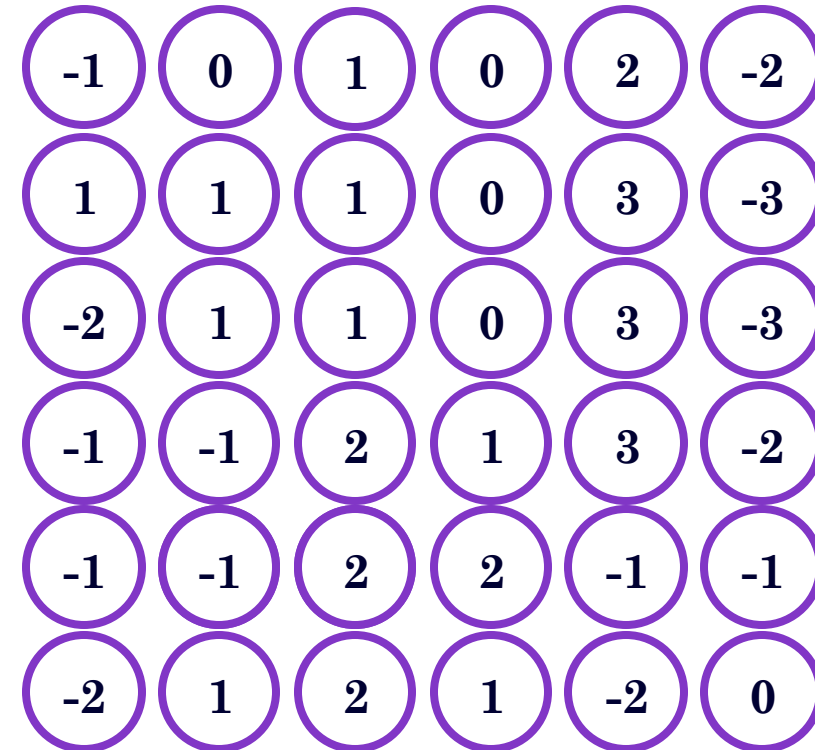
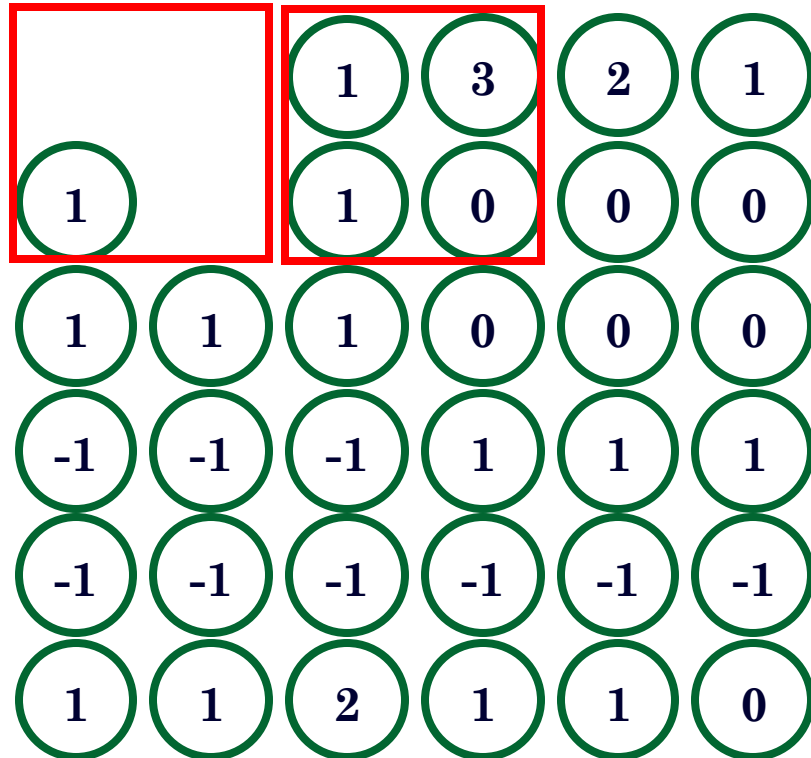
Pooling: downsampling feature maps

2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

“Max pooling”

-1	2	-1
-1	2	-1
-1	2	-1



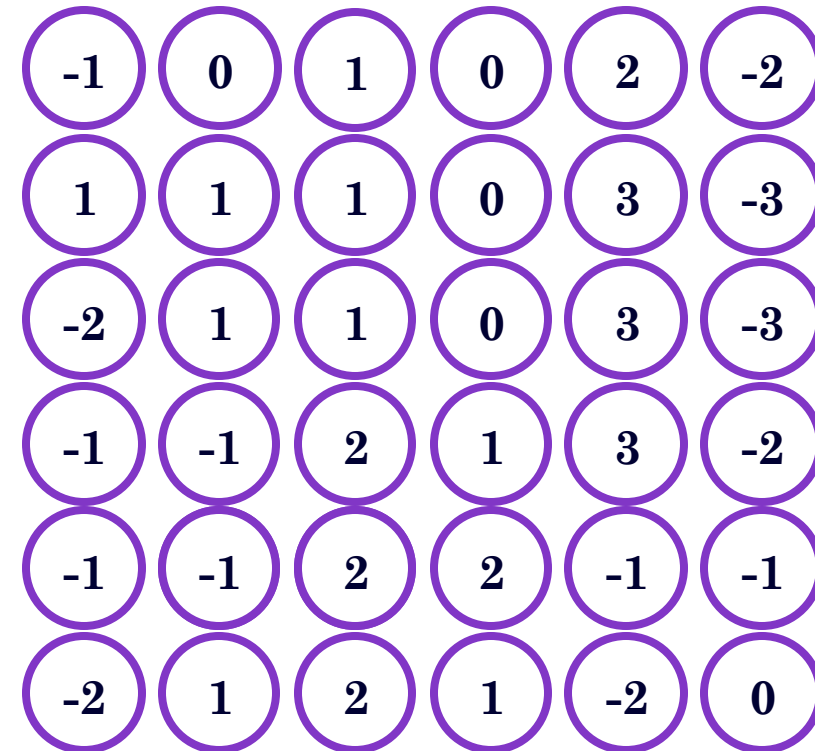
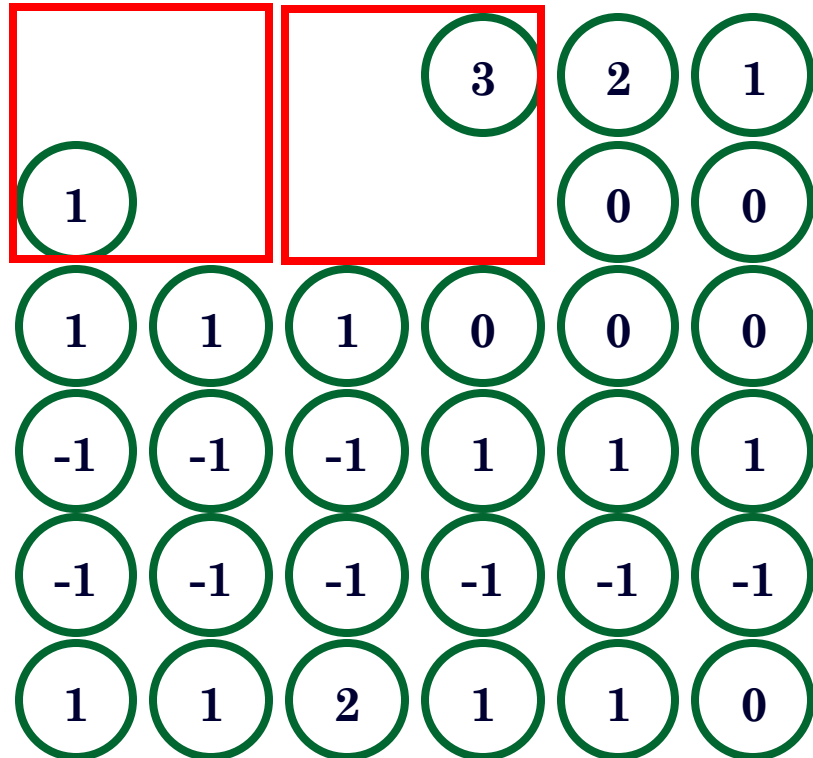
Pooling: downsampling feature maps

2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

“Max pooling”

-1	2	-1
-1	2	-1
-1	2	-1



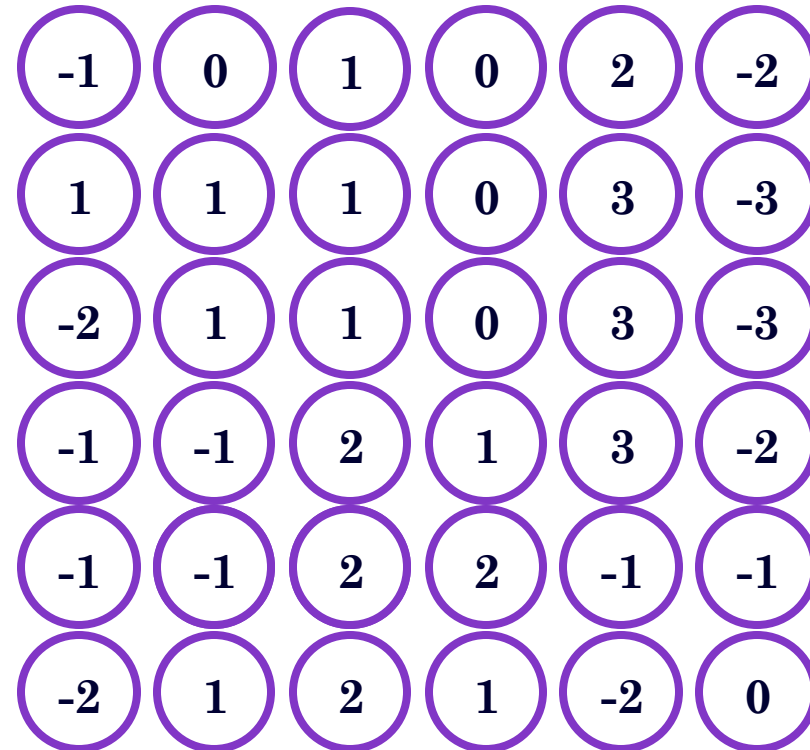
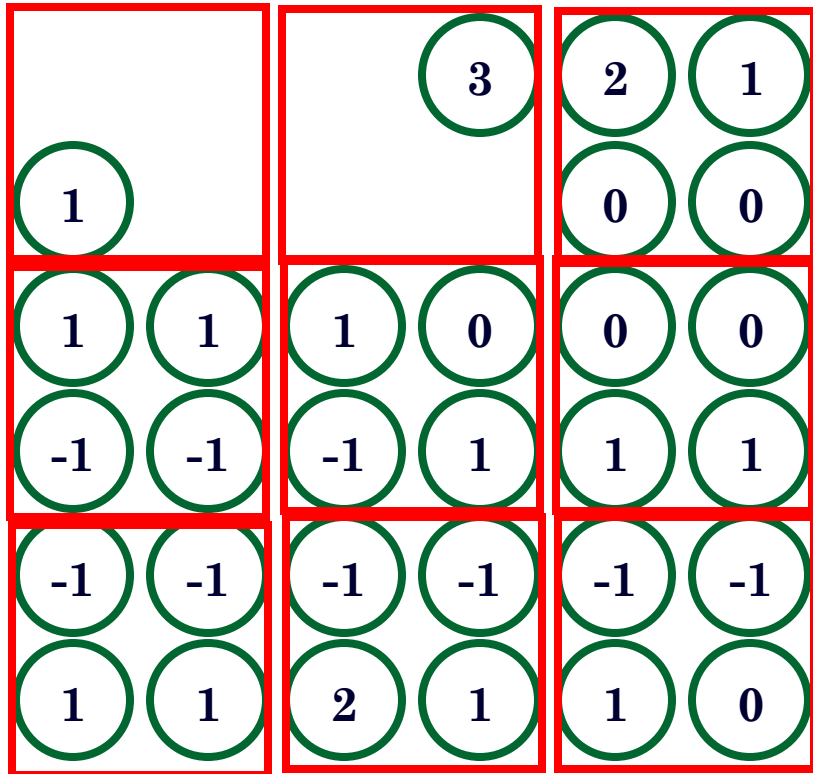
Pooling: downsampling feature maps

2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

“Max pooling”

-1	2	-1
-1	2	-1
-1	2	-1



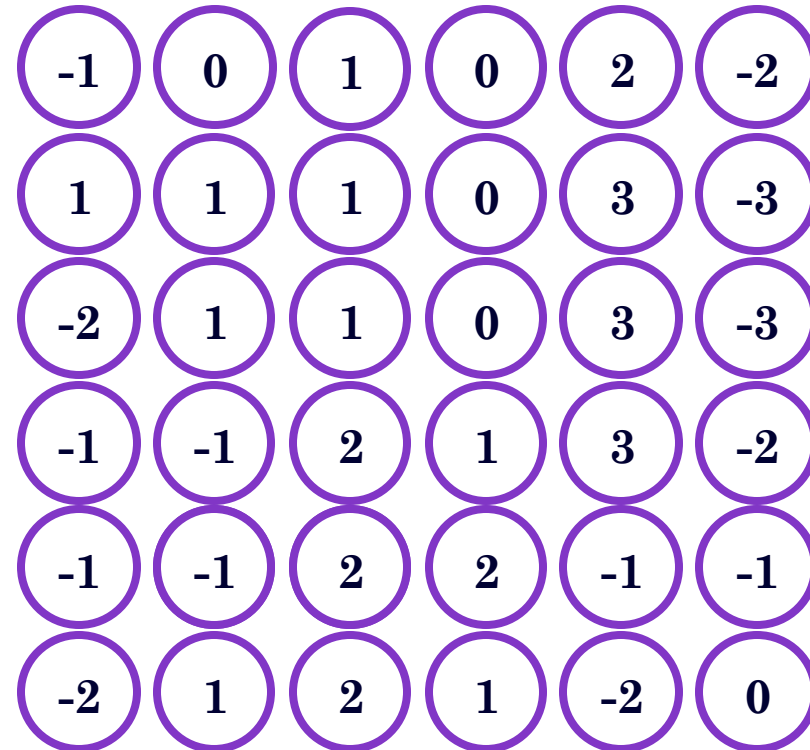
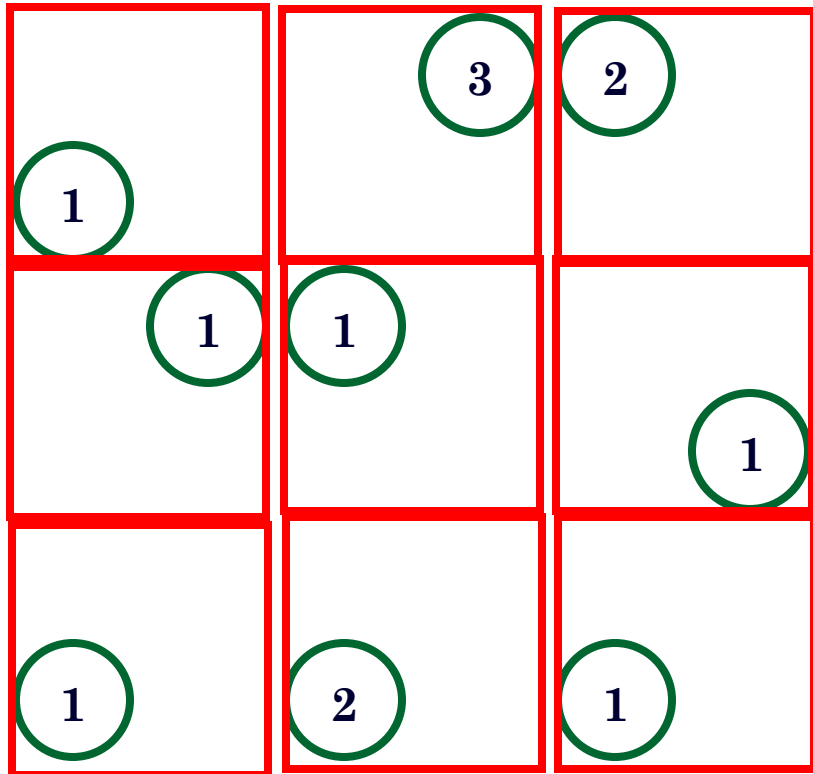
Pooling: downsampling feature maps

2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

“Max pooling”

-1	2	-1
-1	2	-1
-1	2	-1



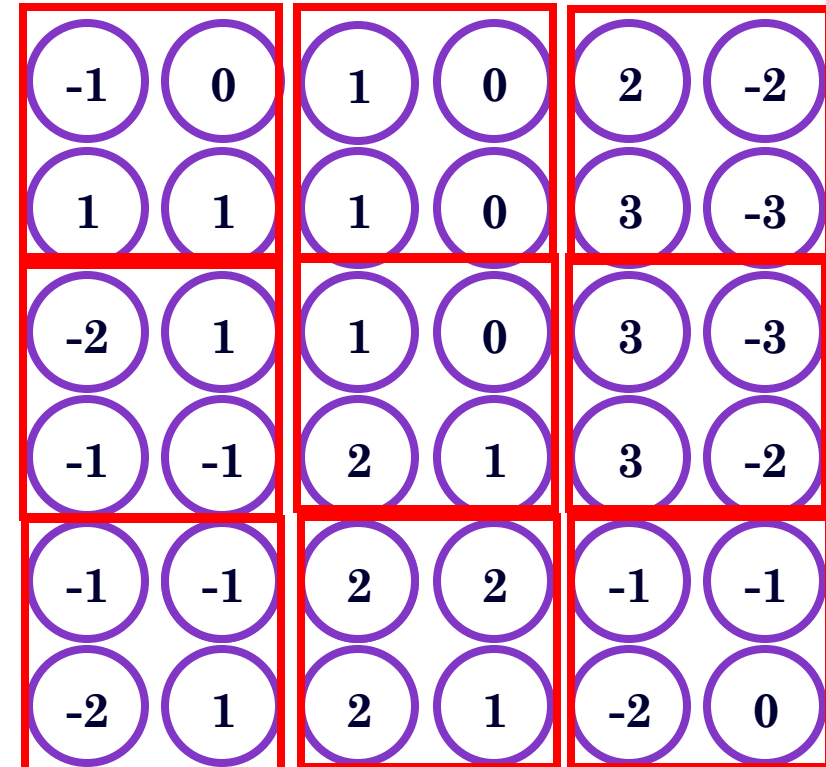
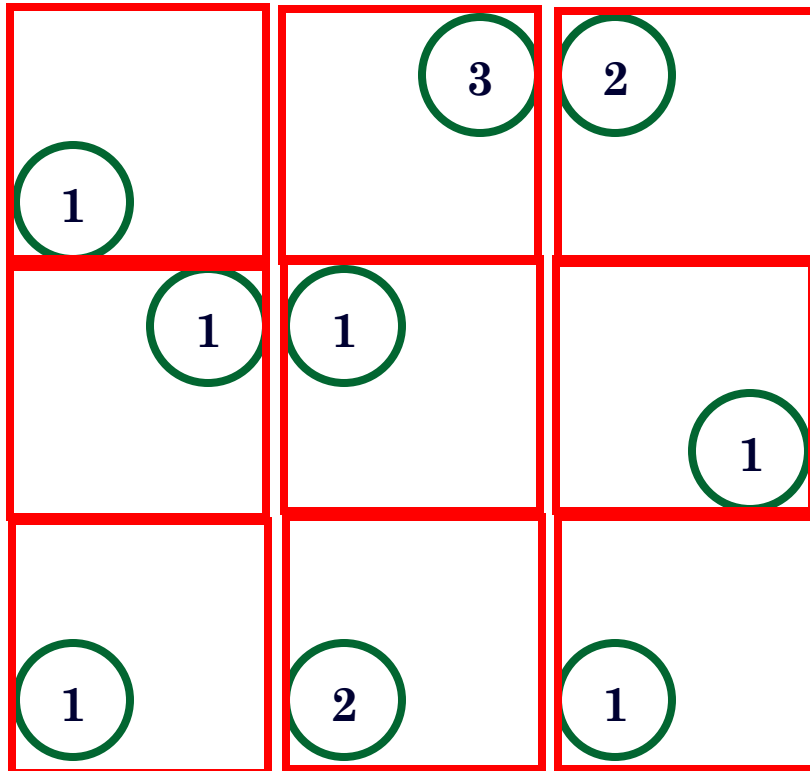
Pooling: downsampling feature maps

2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

“Max pooling”

-1	2	-1
-1	2	-1
-1	2	-1



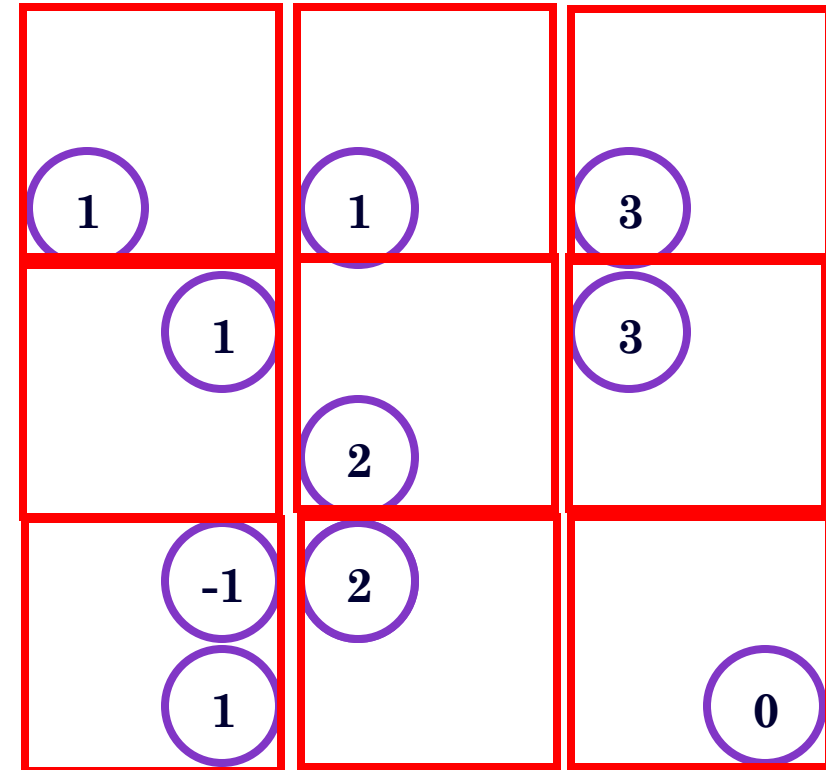
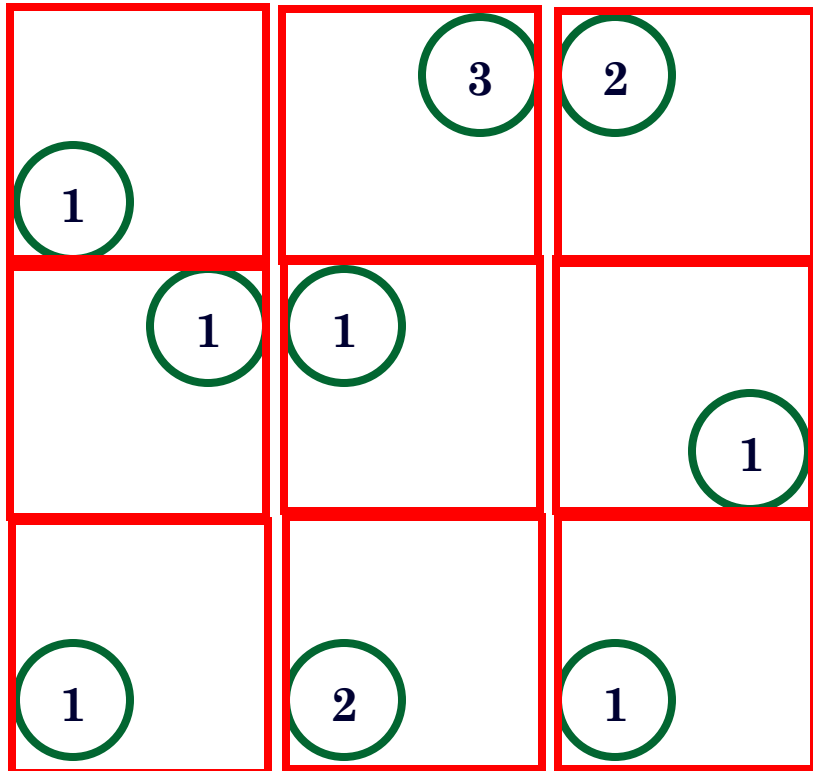
Pooling: downsampling feature maps

2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

“Max pooling”

-1	2	-1
-1	2	-1
-1	2	-1



Pooling: downsampling feature maps

2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

“Max pooling”

-1	2	-1
-1	2	-1
-1	2	-1

1	3	2
1	1	1
1	2	1

1	1	3
1	2	3
1	2	0

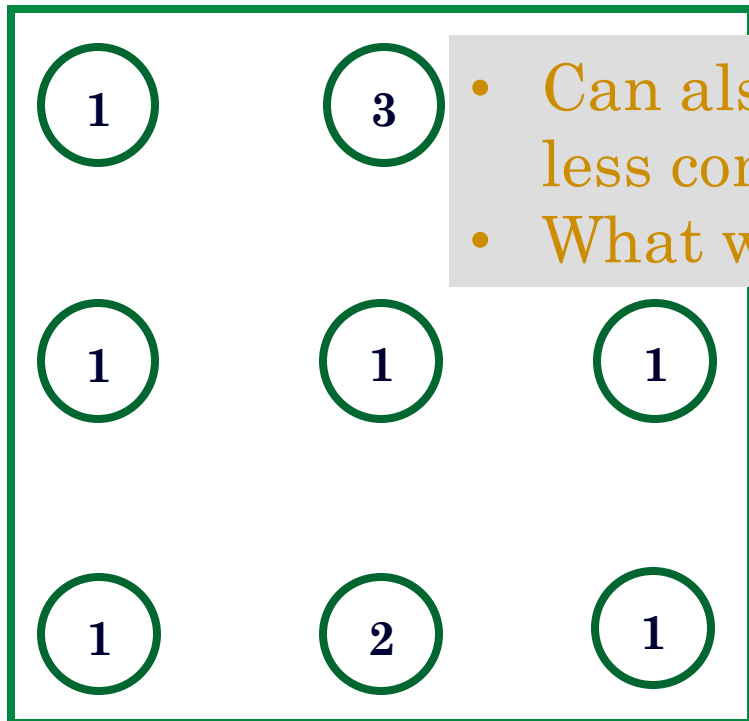
Pooling: downsampling feature maps

2×2 pooling

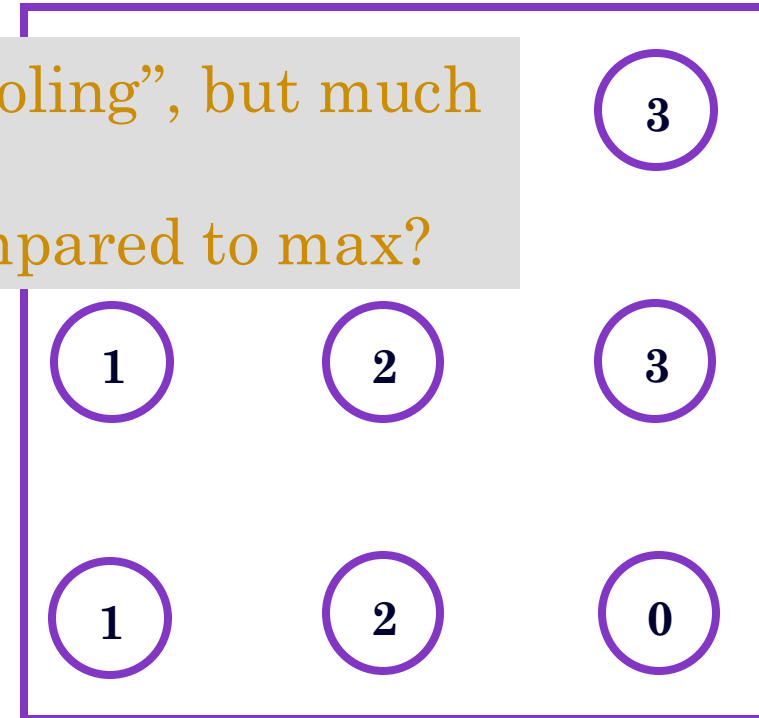
-1	-1	-1
2	2	2
-1	-1	-1

“Max pooling”

-1	2	-1
-1	2	-1
-1	2	-1

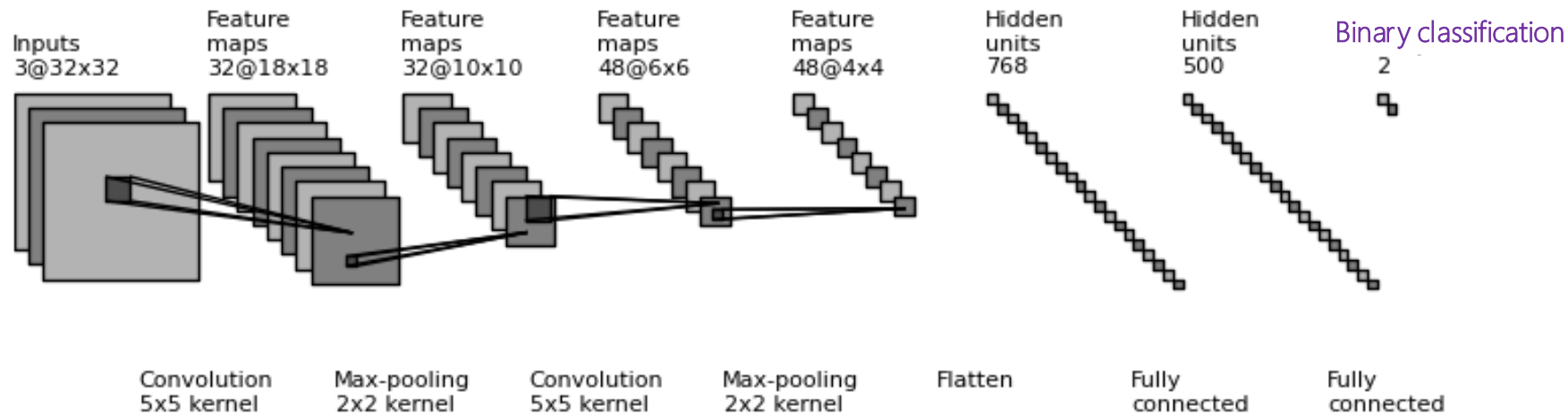


- Can also do “average pooling”, but much less common.
- What would we lose compared to max?

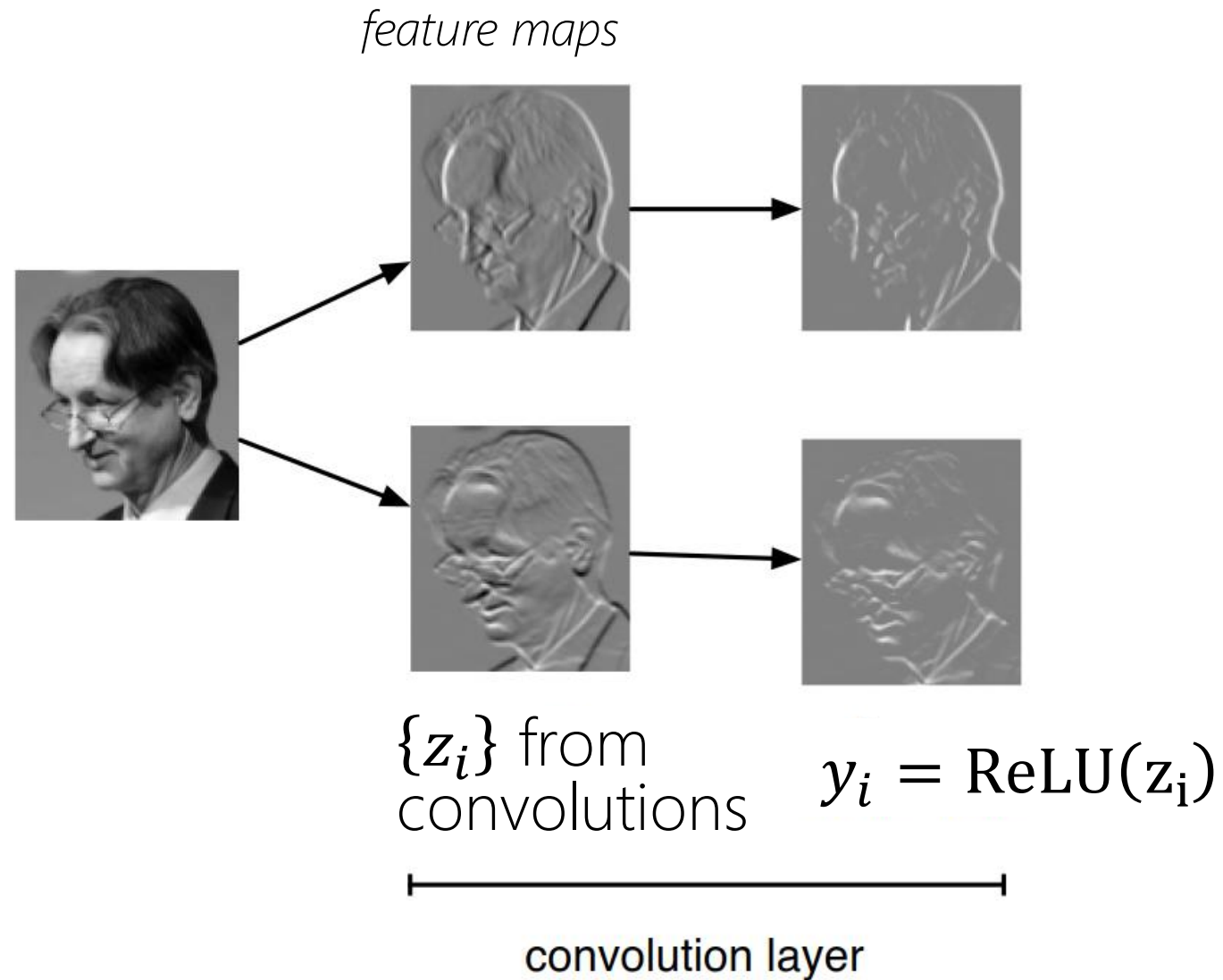


ConvNets: stacking different layers

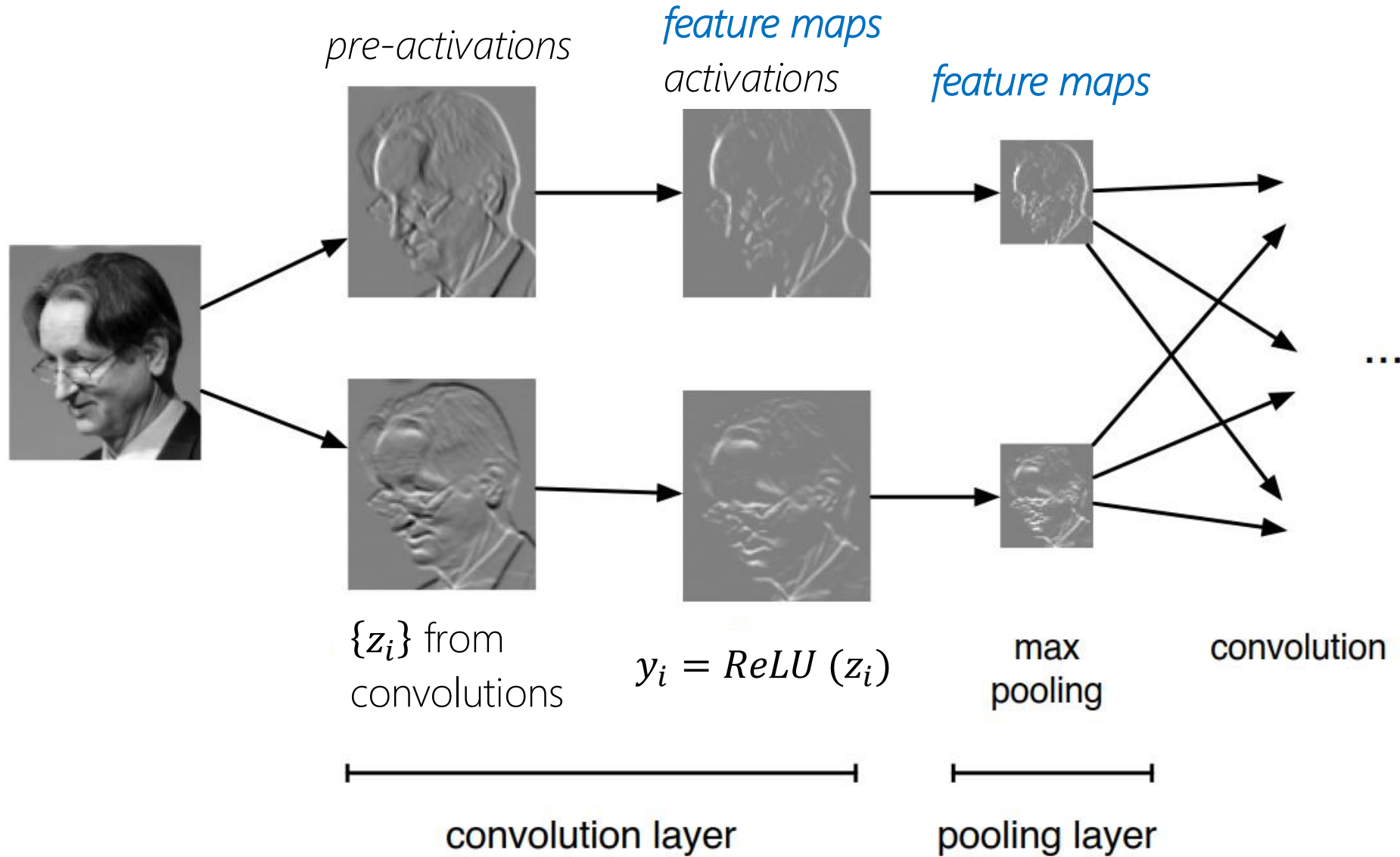
- Convolutional layers: do feature detection.
- Pooling layers: downsample and give local invariance to learned features (aka filters).
- Fully connected help to make final predictions.
- Could have binary/multi-class **classifier** or **regression "head"** (e.g., binomial/Gaussian likelihood, etc.).



Convolutional layer: needs non-linearity

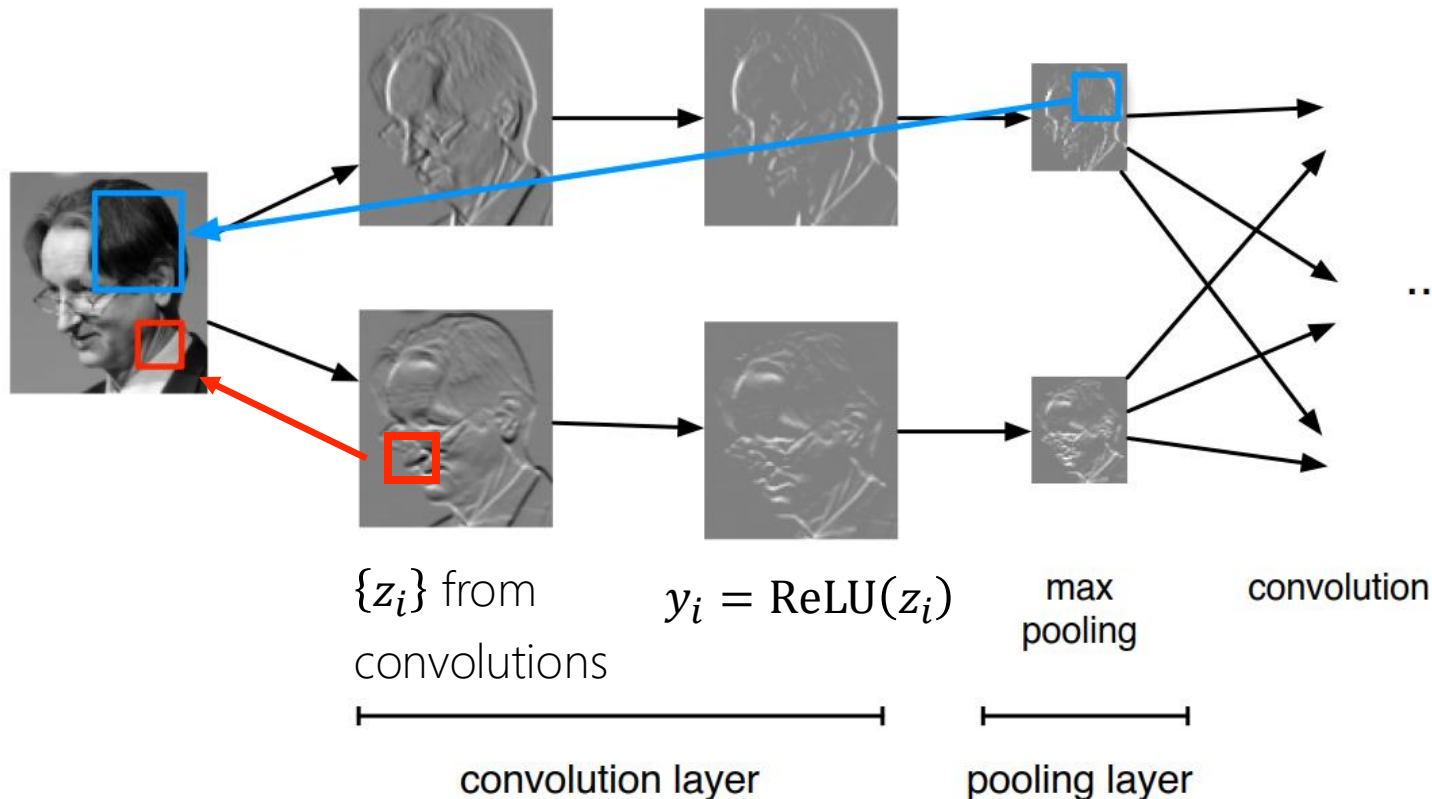


CNNs: convolution + activation + pooling



“Receptive field” of neurons in a CNN

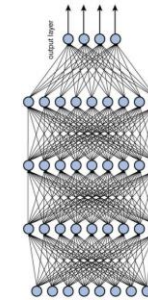
Because of pooling, higher-layer neurons are affected by a larger region of the input than neurons in lower layers, even when filters were the same size.



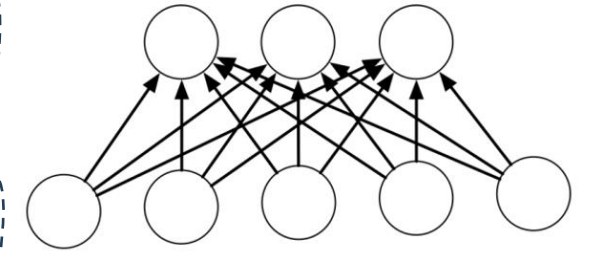
Different strides/filter sizes can also cause this.

Recap: Convolutional NNs (CNNs)

Force the **weights to be shared**.

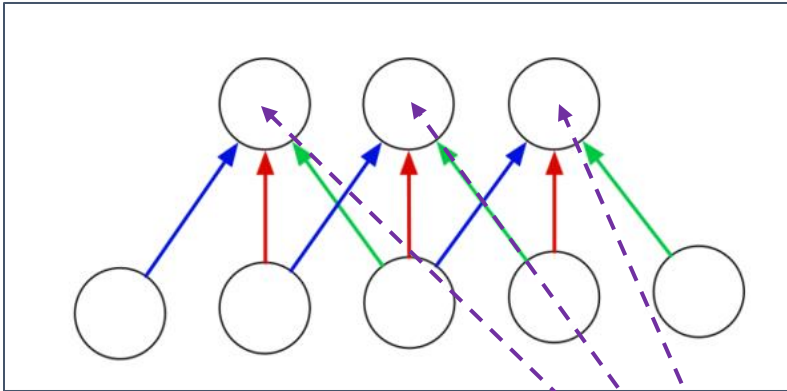


Fully-connected (FC) layer

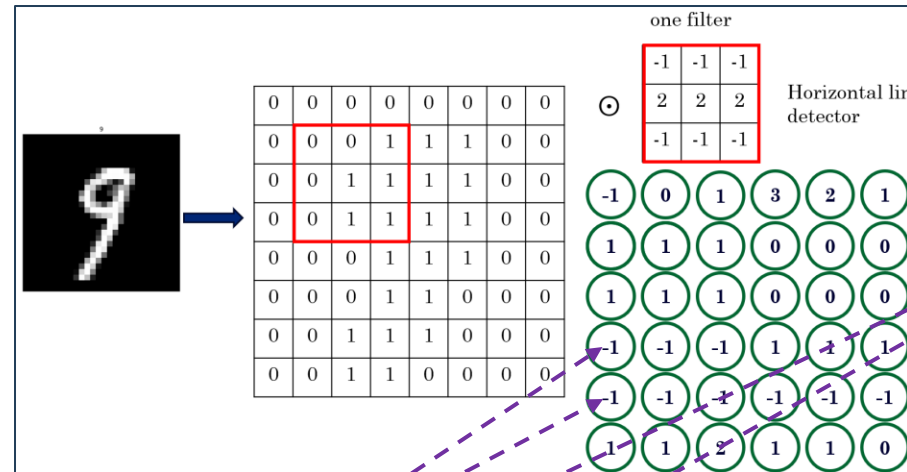


Three ways to depict one convolutional layer:

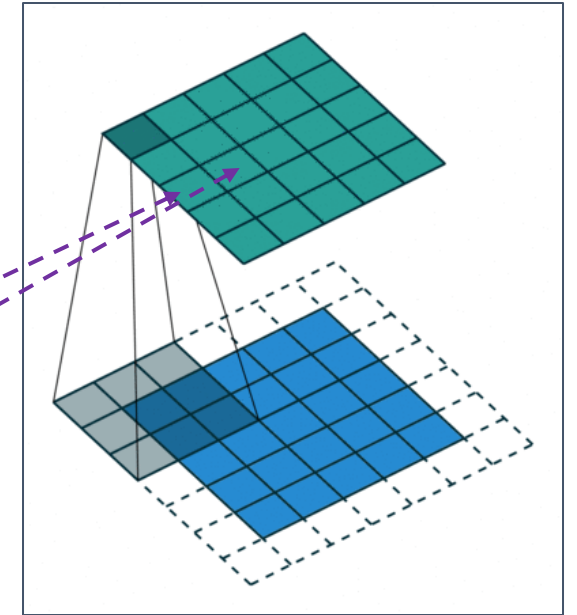
1



2



3



neurons ("hidden units") in a CNN layer
actually include the non-linearity

Training CNNs

- Loss function: maximum likelihood/cross-entropy.
- Gradient descent via backprop, as before:
 - Shared weights (i.e., filters) mean gradients are accumulated across all positions where one 2D filter was applied.
 - Max-pooling: gradient only gets back-propagated through the neuron that “won” the max pool*.

*(technically this is a “sub-gradient”, 1 for winner, 0 for the rest)

Training CNNs: backprop through max pooling

Forward Pass (Max Pooling)

Given a 2×2 region of a feature map:

$$X = \begin{bmatrix} 1 & 3 \\ 2 & 9 \end{bmatrix} \xrightarrow{\text{Max Pool}} Y = 9$$

Because the value 9 was the only element that contributed to the output, it receives the entire gradient. The other elements are "disconnected" from the output, receiving a gradient of 0:

$$\frac{\partial L}{\partial X} = \begin{bmatrix} 0 & 0 \\ 0 & 5 \end{bmatrix}$$

Backward Pass (Gradient Routing)

Assume the loss gradient arriving at this pooling layer from the deeper layers is 5:

$$\frac{\partial L}{\partial Y} = 5$$

v_j is the output node (Y)

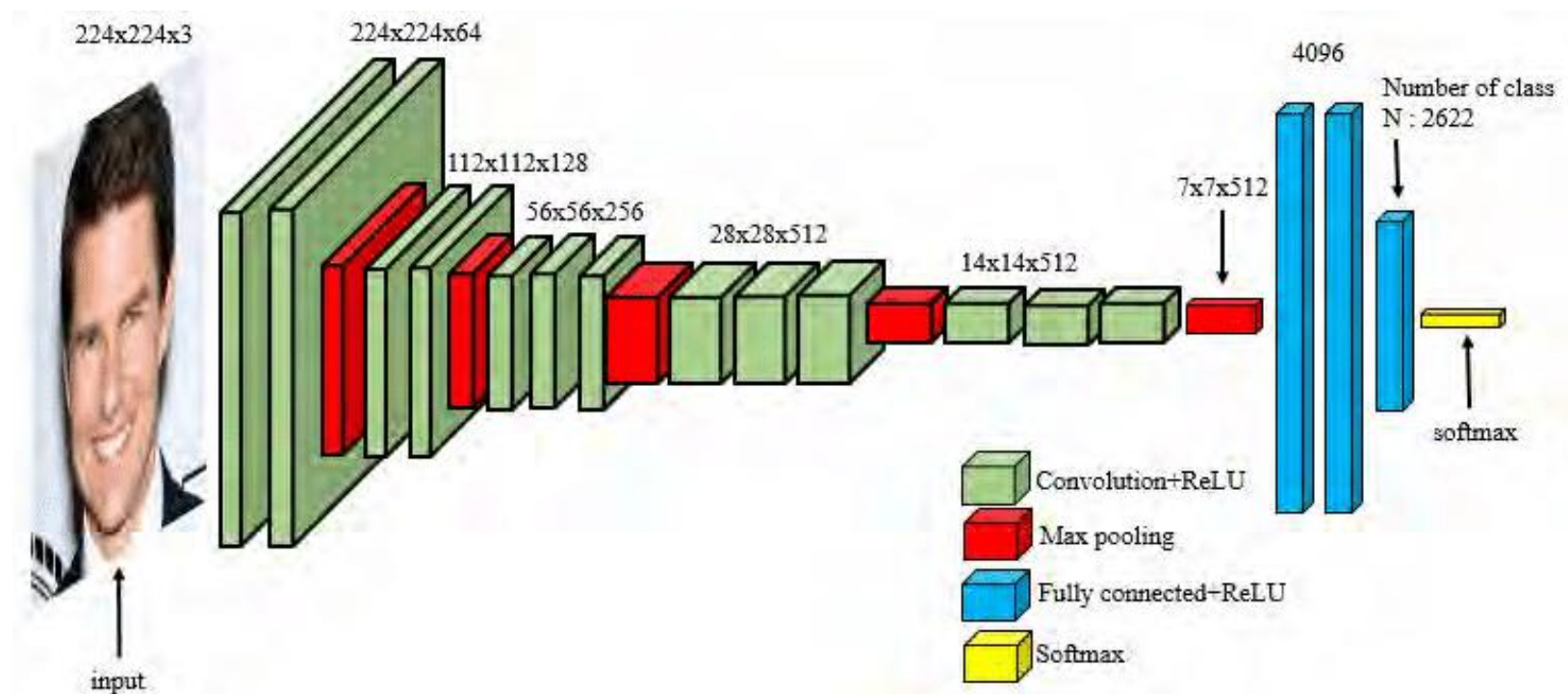
v_i is an input node (an element in X)

$$\overline{v_i} = \sum_{j \in \text{Ch}(v_i)} \overline{v_j} \frac{\partial v_j}{\partial v_i}$$

1 (or 0) for
winner (or loser)

What architecture should we use?

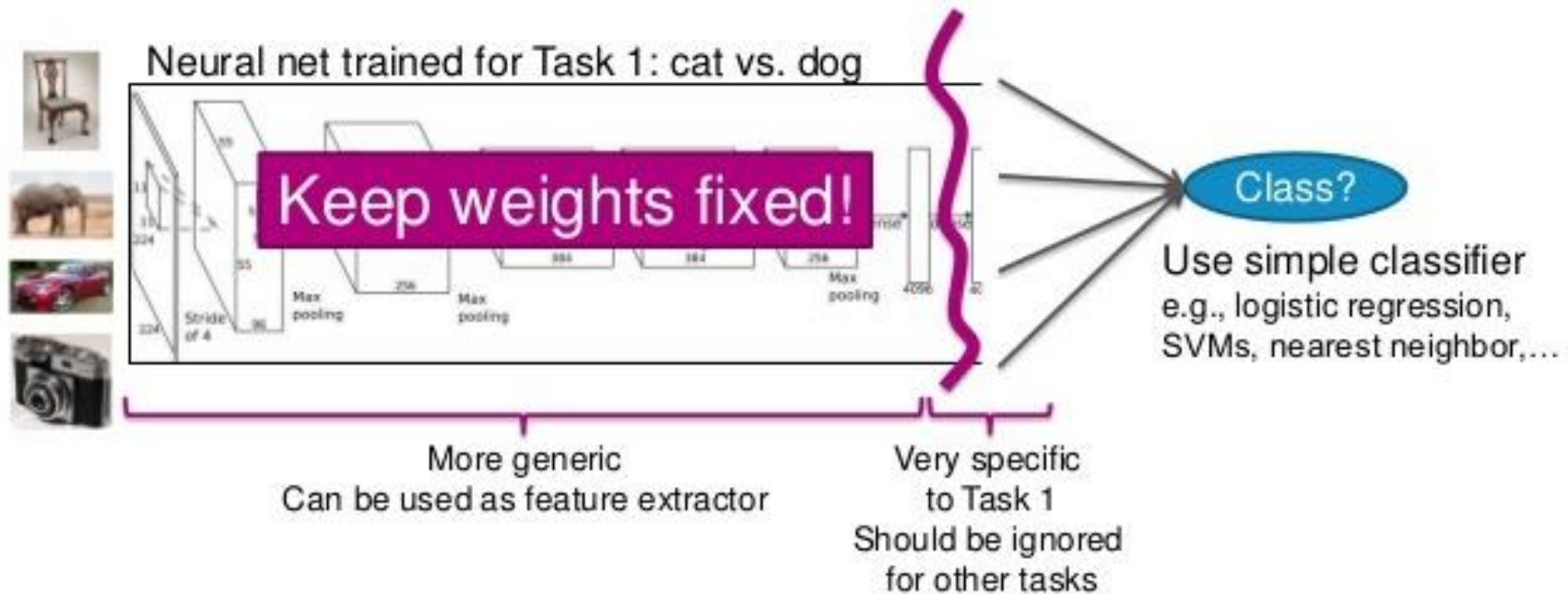
- Use hold out validation (CV too expensive) to figure out a good architecture (depth, width, type of layer,...).
- A well-known network architecture, "VGG(-Face)" from 2014 (millions of parameters)



Re-using a model trained on lots of data

A common thing to do that can work quite well is to:

- i. take a particular architecture that has been developed and fully-trained on some data set
- ii. "Freeze"/fix all the "feature extraction" layer params; re-train only on the top layer(s), which actually do the regression/classification.



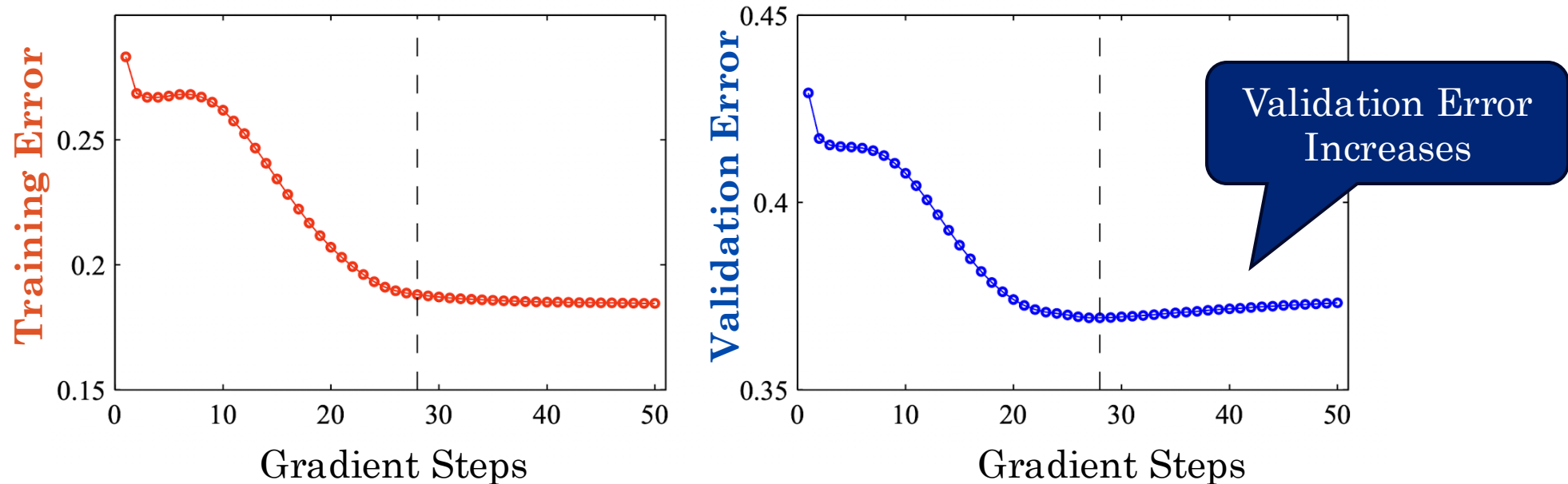
CS 189/289

Today's lecture outline

1. Finish CNNs
2. Other NN stuff

Learning Curves and Early Stopping

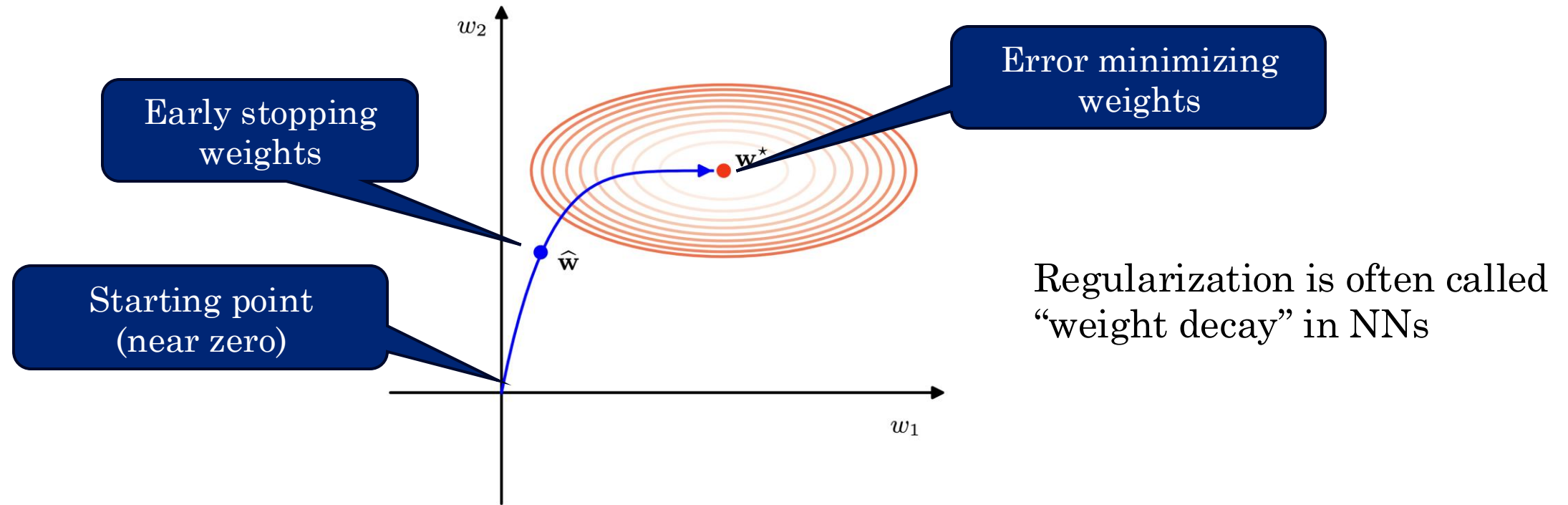
During training it is common practice to plot a **learning curve** which tracks the **training** and **validation error**



Early Stopping: stop training (return to the model checkpoint) when the validation error stopped decreasing.

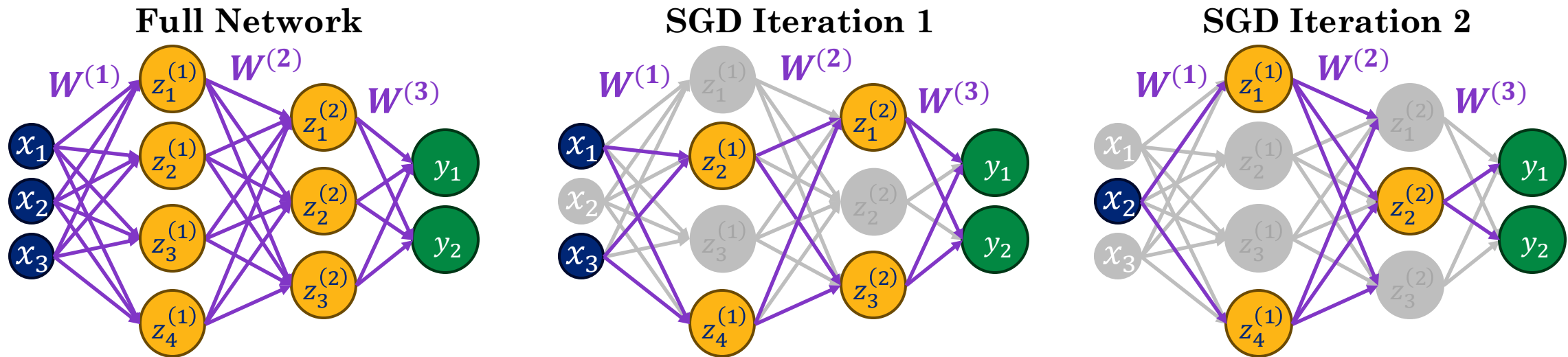
Early Stopping is a form of Regularization

Because models typically start with relatively small weight values, stopping gradient descent early can be viewed as a form of weight decay.



Dropout Regularization

Dropout **regularizes neural networks** by randomly “**disabling**” **neurons** during each iteration of SGD.

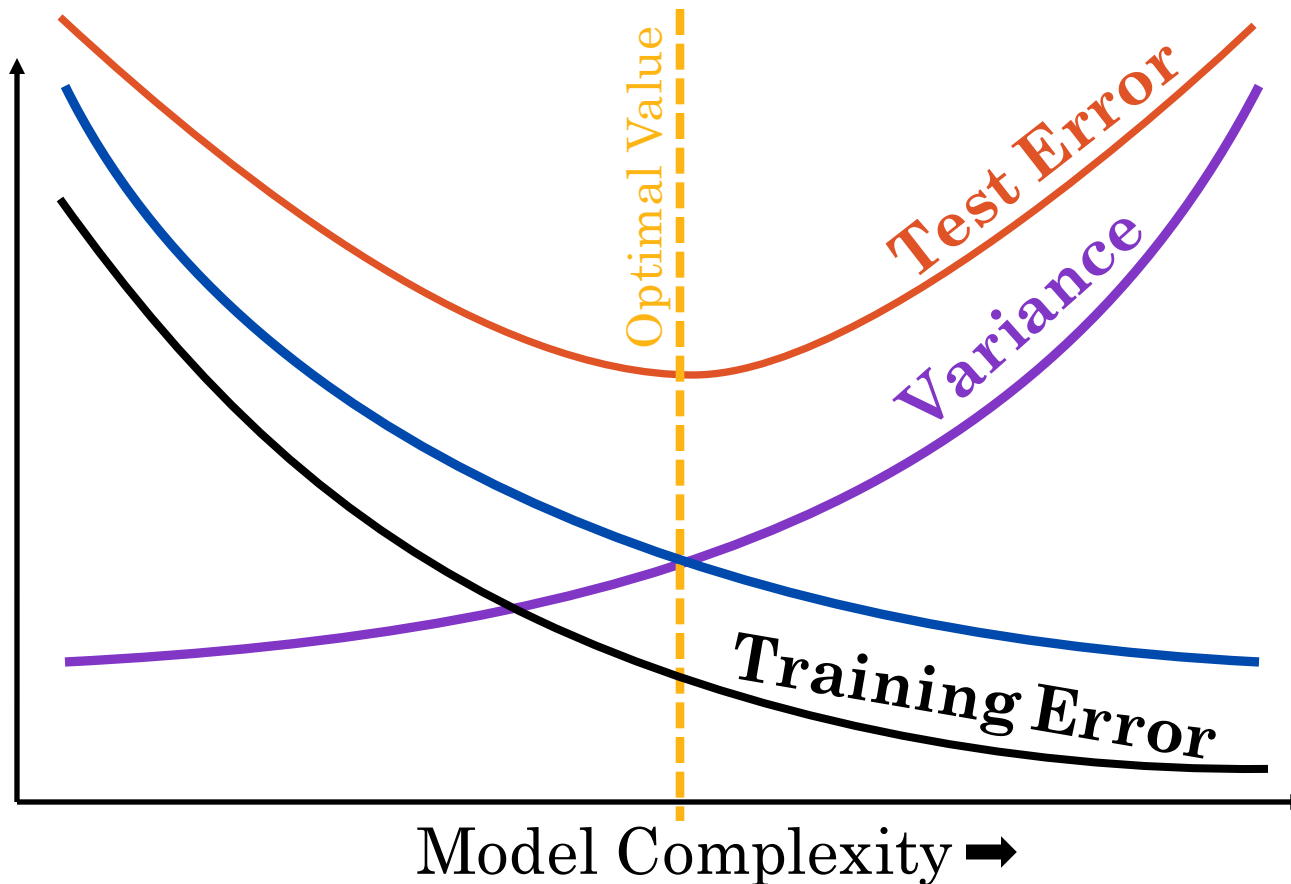


Implemented by sampling $r_m^{(l)} \sim \text{Bernoulli}(1 - \rho)$ and then using $\tilde{z}_m^{(l)} = z_m^{(l)} * r_m^{(l)}$ as the modified activations (typically $\rho \leq .5$).

At **test time** we set $r_m^{(l)} = 1 - \rho$ using all the activations (scaled).

Recall: The Bias Variance Tradeoff

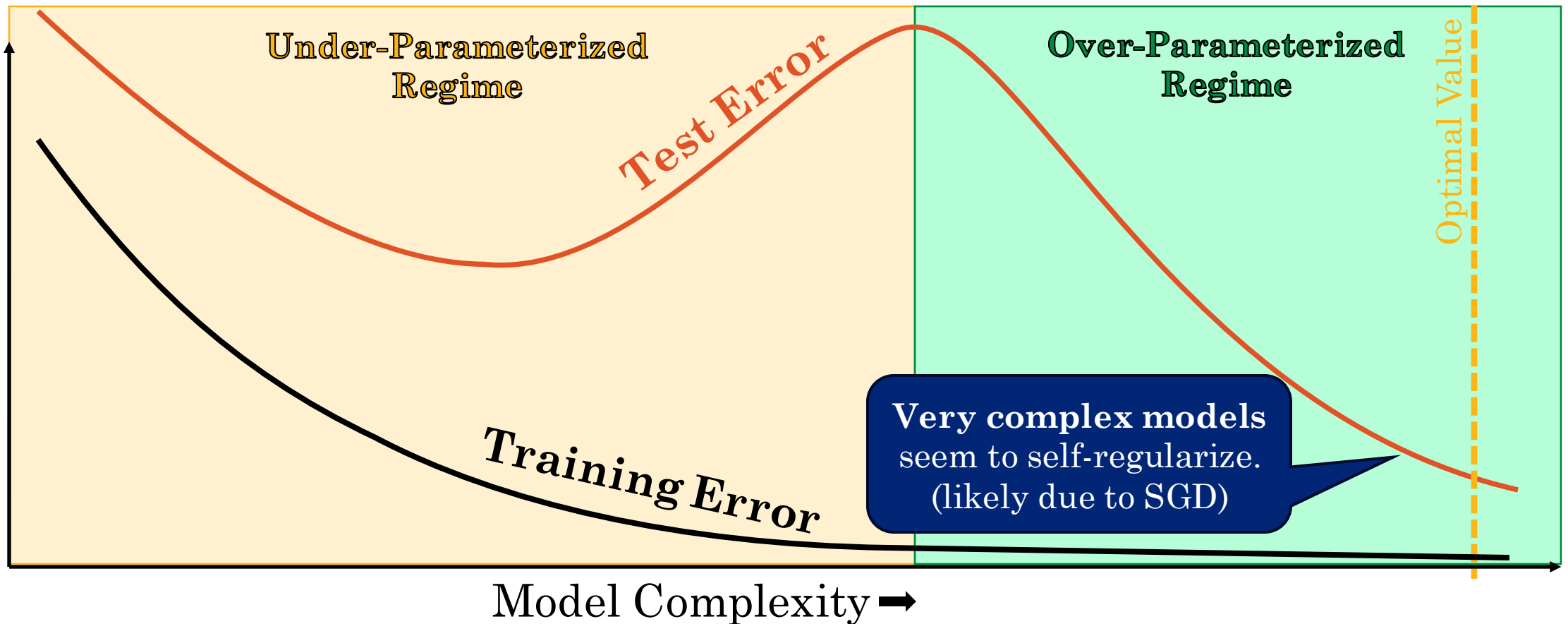
In earlier lectures, we introduced the fundamental **bias-variance tradeoff** and how **bias** and **variance** contribute to **test-error**



$$\begin{aligned} \text{Test Error} &= \\ &\text{Variance} \\ &+ \\ &(\text{Bias})^2 \\ &+ \\ &\text{Irreducible} \\ &\text{noise} \end{aligned}$$

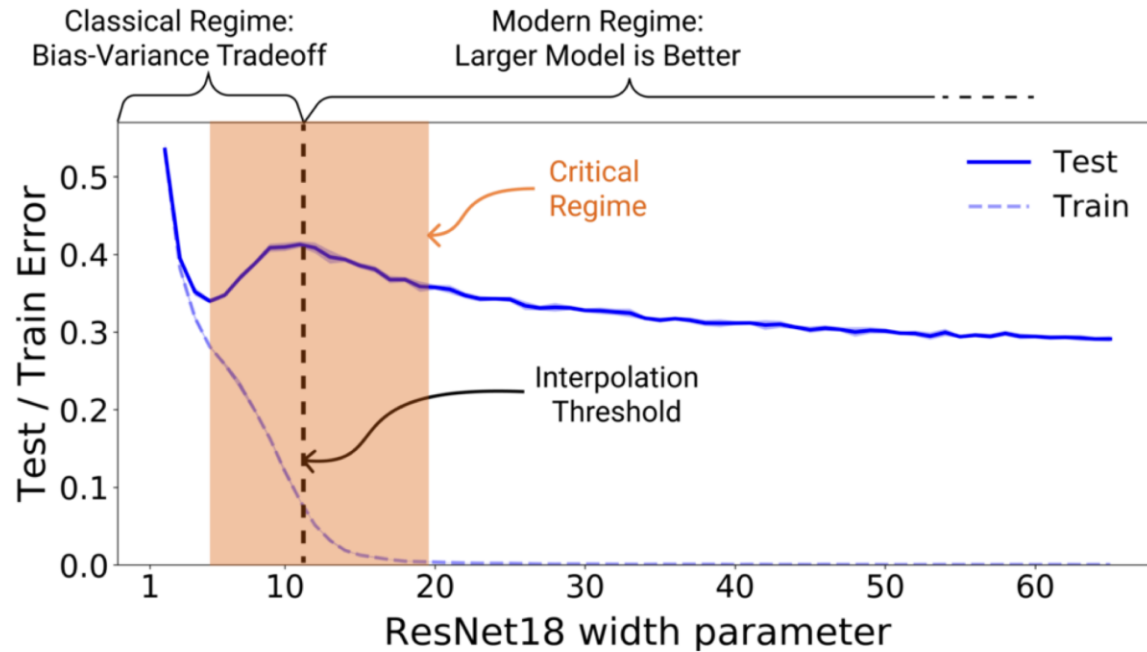
Retake for the modern era: Double Descent

In the case of large **neural networks trained using SGD** there is a **second decrease** in test error.



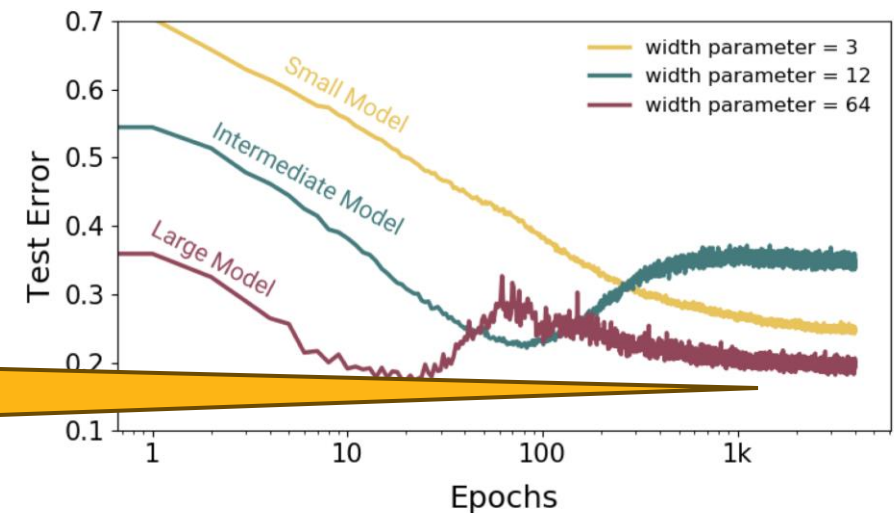
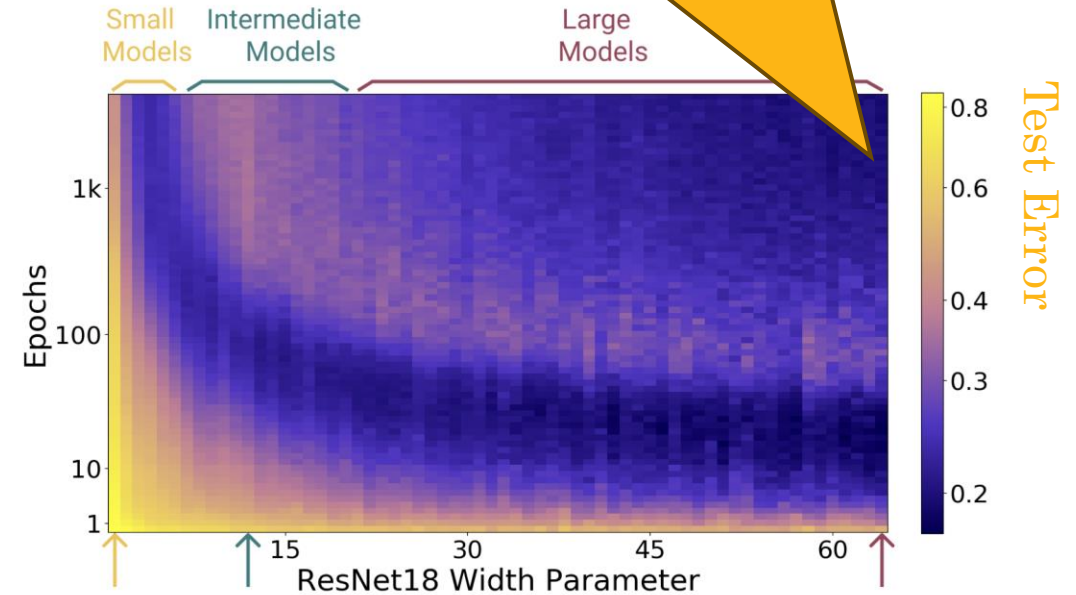
Double Descent in Practice

Examples of double descent
from [Nakkiran et al. ICLR'20](#):



But largest models achieve
lowest test error

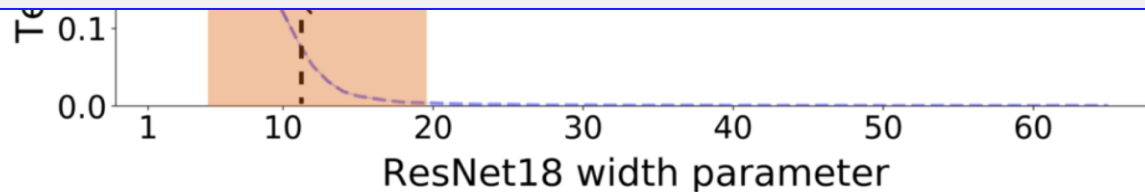
For sufficiently large
models, early stopping could
decrease generalization.



Double Descent in Practice

N.B. You are not responsible for info on this slide as it is quite confusing .

Relative to lecture, I moved this bubble pointer to be higher in top plot so that the bubble make sense and matches the plot below, although to be fair, here it doesn't seem like the second dip is actually lower than the first one, so not a great example, but you get the idea. Also, the bubble pointer is referring to dip in test error along the vertical axis, which is “epoch-wise” double descent, whereas previous discussion was about “model complexity” double descent (along horizontal axis in same plot).



But largest models achieve lowest test error

For sufficiently large models, early stopping could decrease generalization.

